

Ausführlicher Barrierefreiheits-Report mit praxisnahen Handlungsanleitungen

(Prüfbericht-Version 29. 07. 2025 • Grundlage: BIK BITV-Test + WCAG 2.2)

1 | Wahrnehmbar – Alternativtexte & visuelle Kennzeichnung

Problem (WCAG-Kriterium)	Warum ist das kritisch?	„So geht’s“ – Schritt-für-Schritt-Lösung
Funktions-Icons (Profil, Notiz-Toggle, Löschen) tragen gar keinen oder kryptischen <code>alt</code> -Text. (1.1.1 Non-text Content)	Screen-Reader beschreiben nur das Bild („image 123“) – die Funktion bleibt unverständlich.	1. Icon in einen <code><button></code> legen. 2. Aussagekräftiges Label vergeben – am einfachsten via <code>aria-label</code> : <code><button aria-label="Notizen ein-/ausblenden">...</button></code> 3. Das Bild selbst gilt als Deko → <code>alt=""</code> .
Pfeil-Symbole in Links („Zum Kurs →“) werden vorgelesen. (1.1.1)	Das „Arrow right“ stört die eigentliche Link-Ansage.	Symbol als eigenständiges <code>→</code> einfügen oder als CSS-Hintergrund einblenden.
Dekorative Icons (Lupe, Buch-Icon, Raute) erscheinen im Accessibility-Tree. (1.1.1)	Überflüssige Grafik-Ansagen erschweren das Vorlesen.	1. Icons, die via <code>::before</code> gesetzt sind, auf eigenständiges <code></code> verschieben. 2. Spans mit <code>aria-hidden="true"</code> versehen.
Links & Quiz-States werden ausschließlich farblich unterschieden. (1.4.1 Use of Color)	Farbenblinde oder Screen-Reader-Nutzer erkennen Unterschiede nicht.	• Links zusätzlich unterstreichen oder Link-Icon ergänzen. • Quiz: neben Rot/Grün ein Symbol + Text einblenden, z. B. <code>✓ Richtig</code> .
Gelber Text („Anforderungen noch nicht erfüllt“) erreicht nur 1,96 : 1 Kontrast. (1.4.3 Contrast (min))	Nutzende mit Sehschwäche können die Warnung nicht lesen.	a. Gelb abdunkeln – #FFC700 → #866900 (ergibt > 4.5 : 1). b. Alternativ hellen Hintergrund (#FFF) unterlegen. c. Immer mit Kontrast-Checker gegenprüfen.

2 | Wahrnehmbar – Struktur & Semantik

Problem (WCAG 1.3.1)	Warum?	Umsetzung
Notizen-Überschrift liegt als <code><h3></code> IM Button.	Der Button „überschattet“ die Überschrift – Screen-Reader interpretieren sie nicht mehr.	Konstruktion tauschen: <code><h3><button aria-label="Notizen">...Notizen</button></h3></code>

Problem (WCAG 1.3.1)	Warum?	Umsetzung	
Modul-Navigation und Kurslisten fehlen als echte Liste.	Ohne <code></code> fehlt die Relation „Element von Gruppe“.	<code><nav aria-label="Modulnavigation"><a>...</code>	
Testimonials nicht als Zitat.	Screen-Reader erkennen keinen Rollenwechsel.	<code><blockquote lang="de"><p>...</p><cite>Person XY</cite></blockquote></code>	
Dokument-Titel lautet nur „Modul 1“.	Braille-Zeilen & Browser-Tabs ergeben wenig Kontext.	Im <code><title></code> -Tag: `Modul 1 - Grundlagen`	tollwerk Academy`

3 | Bedienbar – Tastatur & Fokus

Problem (WCAG)	Wirkung	Konkrete Lösung	
Kurs-Teaser-Cards nicht fokussierbar. (2.1.1 Keyboard)	Tastatur-User können Kurse nicht öffnen.	Card-Muster: <code><article><h3>...</h3>Zum Kurs</article></code> (<code>position:relative</code> + <code>.stretched-link{...}</code>)	
Schalter „Notizen anzeigen“ ohne sichtbare Fokus-kontur. (2.4.7 Focus Visible)	Unklar, wo man sich befindet.	Globale Regel: <code>button:focus-visible{outline:2px solid currentColor;outline-offset:4px}</code>	
Fokusverlust nach Lösch-Aktion in Notizen. (3.2.1 On Input)	Tastatur hängt am Seitenanfang.	Nach <code>removeChild()</code> per JS: <code>(prevEl</code>	<code>addBtn).focus();`</code>
Deaktivierte Buttons bleiben fokussierbar. (2.1.1)	„Dead ends“ im Tab-Verlauf.	Entfernen: <code>disabled</code> + <code>tabindex="-1"</code> + CSS-Stil.	

Problem (WCAG)	Wirkung	Konkrete Lösung
Scroll-Container erhält schlecht sichtbaren Focus. (2.4.7)	Kaum wahrnehmbar.	<code>:focus-visible{box-shadow:0 0 0 3px #005fcc inset}</code> oder Scrollbarkeit entfernen.
Fortschritts-Animation läuft endlos. (2.2.2 Pause, Stop, Hide)	Kann ablenken oder Epilepsie triggern.	CSS: <code>animation-iteration-count:1</code> oder Play/Pause-Schalter implementieren.

4 | Bedienbar – Navigation & Landmark-Regeln

Problem (WCAG 2.4.1)	Warum?	Lösung
Redundante Landmarks (<code><aside></code> + <code>role="region"</code> im <code><nav></code>).	ARIA-Tree wird unübersichtlich.	Überflüssige Elemente schlicht entfernen.
Englische Landmark-Labels („Content progress“).	Sprachausgabe mischt Englisch ein.	<code>aria-label="Inhaltsfortschritt"</code>
Tab-Navigation (Video/Audio/Text) ignoriert ARIA-Tabs-Pattern.	Screen-Reader melden keine aktive Registerkarte.	<code><div role="tablist"></code> + Buttons <code>role="tab"</code> • <code>aria-selected="true"</code> auf aktivem Tab. - Panel bekommt <code>role="tabpanel"</code> + <code>tabindex="0"</code>.

5 | Verständlich – Formulare & Fehlermeldungen

Problem (WCAG)	Wirkung	Schritt-weise Korrektur
Suchfeld ohne sichtbares Label & Submit-Button. (3.3.2 Labels)	Screen-Reader erkennen Feld nicht; Maus-User haben keinen Auslöser.	```html
Seite durchsuchen		

6 | Robust – Name / Role / Value & valider Code

Problem	Was passiert?	Best Practice
Notiz-Button trägt gleichzeitig <code>role="heading"</code> .	Element verliert Schalter-Rolle.	Überschrift außerhalb des Buttons platzieren (s. Abschnitt 2).

Problem	Was passiert?	Best Practice
Nested Links im Button (interaktive Kachel).	Doppelte Ziel-Regionen → Fehlbedienung.	Schalter UND Link voneinander trennen; nur eins pro Interaktions-Fläche.
Medien-Tabs ohne States.	ARIA-States fehlen, Inhalt wird nicht zugeordnet.	Vollständiges Tabs-Pattern (siehe Abschnitt 4).

Priorisierte Roadmap

1. **Blocker („must fix“)** \ Alt-Texte, Tastaturfokus, Labels, Kontrast $\geq 4.5 : 1$, redundante Landmarks.
2. **Sichtbarkeit & Orientierung** \ Fokus-Stile, farbumabhängige Zustände, eindeutige Überschriften / Listen.
3. **Interaktive Komponenten** \ Tabs-Pattern, Fortschritts-Animation, Card-Muster & Fokus-Logik.
4. **Fehlerführung & Feinheiten** \ Feldnahe Alerts, konsistente Region-Labels, Doku-Titel, Blockquotes.

Arbeitsschritte effizient organisieren

Sprint-Task	Check	WCAG-Ref.
<i>Icon-Buttons refactor</i>	Visuell & Screen-Reader richtig?	1.1.1
<i>Kontrastpaletten definieren</i>	$\geq$ AA-Kontrast?	1.4.3
<i>Globales Focus-CSS</i>	Gut sichtbar auf allen Komponenten?	2.4.7
<i>ARIA-Tabs implementieren</i>	Rollen + States vorhanden?	4.1.2
<i>Fehler-Alerts</i>	Live-Regions & Fokus?	3.3.1

Lessons Learned – Reflexion unserer Barrierefreiheits-Prüfung

1. Accessibility-First-Mindset

- **Frühzeitige Einbindung:** Wir haben erkannt, dass Barrierefreiheit bereits in der Konzept- und Designphase bedacht werden muss, um kostspielige Nacharbeiten zu vermeiden.
- **WCAG als roter Faden:** Künftige Stories und Akzeptanzkriterien verankern stets das passende WCAG-Level.

2. Non-Text-Content: Alt-Texte & Dekoration

- **Funktion vs. Dekoration:** Funktionale Bilder erhalten aussagekräftige `aria-label`s; rein dekorative Elemente werden konsequent mit `alt=""` bzw. `aria-hidden="true"` versteckt.
- **Pattern-Bibliothek:** Für Icon-Buttons legen wir jetzt ein wiederverwendbares Code-Snippet an, das korrekte Beschriftung sicherstellt.

3. Struktur & Semantik

- **Übersicht durch Überschriften:** Eine strenge H-Hierarchie sorgt für logische Gliederung – Headings gehören nie in interaktive Elemente.
- **Listen & Landmarks:** Wir setzen `<ul>`, `<nav>` und weitere Landmark-Rollen nur dort, wo sie wirklich Mehrwert bieten.

4. Tastatur-Bedienbarkeit & Fokusmanagement

- **Klarer Fokus:** Sichtbare Focus-Konturen und sinnvolle Tab-Reihenfolgen erhöhen die Orientierung.
- **Fokusbeibehalt:** Nach dynamischen DOM-Änderungen wird der Tastaturfokus bewusst neu gesetzt, um "Verloren-gehen" zu verhindern.

5. Farbkontrast & Farbunabhängigkeit

- **Kontrast-Checker:** Wir nutzen automatisierte Tools (z. B. axe DevTools), um Zielwerte $\geq 4.5:1$ sicherzustellen.
- **Redundante Signale:** Richtig/Falsch-Status und Links werden zusätzlich mit Text oder Symbolen ausgezeichnet.

6. ARIA-Patterns & Interaktive Komponenten

- **Tabs, Toggles & Co.:** Wir halten uns strikt an die WAI-ARIA Authoring Practices, um Name/Role/Value konsistent bereitzustellen.
- **Keine Nested Interactives:** Ein Element = eine Rolle = eine Aktion.

7. Formulare & Fehlermeldungen

- **Label-Pflicht:** Jedes Eingabeelement erhält ein sichtbares oder off-screen-Label.
- **Live-Regions:** Fehlermeldungen werden als `role="alert"` ausgegeben und der Fokus springt zur Fehler-Übersicht.

8. Animation & Bewegung

- **Zeitbegrenzung:** Animierte Elemente enden nach max. 5 s oder können pausiert werden.
- **Reduziert-Bewegung-Query:** Wir respektieren `prefers-reduced-motion`.

9. Test-Workflow & Toolchain

- **Automatisierte Checks:** axe-CI-Linting in Pull Requests verhindert Regressionen.
- **Manuelle Prüfungen:** Jede User-Story wird mit NVDA (Windows) und VoiceOver (macOS) keyboard-only getestet.
- **Dokumentation:** Jedes A11y-Issue verlinkt auf das betreffende WCAG-Kriterium.

Was wir bereits gut gemacht haben

- **Konsistente, großflächige Buttons** mit mindestens 44×44 px Zielgröße.
- **Klare Seitennavigation** mit eindeutigem Seiten-Titel und Breadcrumb.
- **Responsives Layout**, das auch bei 400 % Zoom ohne horizontales Scrollen nutzbar ist.

- **Solides HTML-Grundgerüst** (`<!DOCTYPE html>`, `lang`, `charset`, `viewport` korrekt gesetzt).